As you implement the step methods for this scenario, you'll use the scenario example to drive out the features you need from the next layer down.

This process doesn't stop with the step implementation. When you write the production code, you can do exactly the same thing, discovering what services you need from other components (which may not exist yet), and writing the code you'd like to have. This act of imagining the code that would serve you best is a powerful design practice: what information do you need to get the job done? What services do you need to call? Do they exist, or do they need to be created? What would be the most convenient way to obtain the data you need, and in what form? When you write this *imaginary* code, you're providing an example of how you'd like to use an API that doesn't yet exist. Because you haven't written any implementation code yet, this example will not be polluted by preconceived ideas about what the technical solution should look like; rather, it will be driven by what would be easiest to use from the perspective of the developer using the API.

In this way, BDD at the requirements level (where you use more communication-focused tools, such as Cucumber and JBehave) flows naturally to BDD at a more technical level (where you often use tools more traditionally associated with unit testing). We'll study this lower-level form of BDD in much more detail in the next chapter.

9.6 Summary

In this chapter you learned about different sorts of non-UI acceptance testing, including

- When you should use UI tests and when it's more appropriate to use non-web tests
- How to write BDD scenarios that bypass the UI and exercise the controller layer, service layer, or business logic directly
- How to write BDD scenarios for remote services such as web services
- How to use BDD scenarios to describe and verify nonfunctional requirements, such as performance
- How BDD techniques at the requirements level help encourage clean, well designed APIs, and lead naturally to lower-level BDD practices at the unit- and integration-testing level

In the next chapter, you'll learn how BDD practices are also applicable at the coding level, and how this sort of lower-level BDD relates to Test-Driven Development and more traditional unit-testing practices.